

Agmarknet Scraper — Site-Break Fix, Date Backfill, and VM Deployment

What broke, what was fixed, and how the scraper ended up running as an always-on service on a separate VM instead of a console window on a dev laptop. Written 2026-08-11, the day all of this happened.

1. What triggered this

`last_known_prices.json` had gone stale — the newest data in it was dated 07-08-2026, four days behind. `bot_processors/pricing/scraper_daemon.py` 's own log (`scraper_daemon_log.txt`) showed every run since 2026-08-10 dying the same way:

```
playwright._impl._errors.TimeoutError: Page.click: Timeout 30000ms exceeded.  
Call log:  
  - waiting for locator("text=Price & Arrival Reports")
```

Agmarknet had restructured its homepage nav sometime around 2026-08-10 — the `text=Price & Arrival Reports` link `_open_daily_price_form()` clicked through no longer existed, so every scheduled scrape had been silently navigating nowhere for days with nobody noticing.

2. Fixes in `agmarknet_scraper.py`

2.1 Navigation

`_open_daily_price_form()` used to click through the homepage (`Price & Arrival Reports` → `Daily Price and Arrival`). Replaced with a direct `page.goto()` to the form's own stable URL, `https://agmarknet.gov.in/daily-price-and-arrival-report` — simpler and immune to further homepage nav reshuffles.

2.2 Two pre-existing bugs in the standalone entry point

Found while getting `python -m bot_processors.pricing.agmarknet_scraper` (the documented manual-run command) working again — both predate the site restructure, just never triggered because nobody had run this entry point in a while:

- **.env loaded too late.** `db_pool.py` reads `DATABASE_URL` from the environment at *import time*. `__main__` 's `load_dotenv()` call ran *after* the top-of-file `from bot_processors.core.db_pool import get_pool` had already executed, so `DATABASE_URL` came back empty. Fixed by moving `load_dotenv()` to the top of the file, before that import — matches the pattern `scraper_daemon.py` already used correctly.
- **Playwright sync API called inside a running asyncio loop.** `scrape_all()` is synchronous (drives a real `sync_playwright` browser). `__main__` called it directly from inside `asyncio.run(...)`, which Playwright's sync API refuses outright. Fixed by wrapping it in `asyncio.to_thread(scrape_all, ...)`, again matching `scraper_daemon.py` 's existing pattern.

2.3 New capability: scraping a specific past date

The rendered form has **no date field at all** — every query it fires hardcodes `from_date / to_date` to today, with no UI to change that. Inspecting the actual network request the "Go" button sends (not the visible form) found the backend honors arbitrary past dates when asked — confirmed by fetching real 09-08-2026 rows through this same mechanism while the live site only showed the 11th.

`_install_date_override(page, target_date)` uses `page.route()` to intercept the POST to `api.agmarknet.gov.in/v1/daily-price-arrival/report` and rewrite its `from_date / to_date` fields before it's sent. A no-op when `target_date` is today, so the default path is unchanged.

`scrape_all()` gained a `target_date: date | None = None` parameter (defaults to today), and `__main__` gained a `--date=YYYY-MM-DD` flag:

```
python -m bot_processors.pricing.agmarknet_scraper --date=2026-08-09
```

Caveat: only verified for dates a few days back (8th/9th against an 11th "today"). How far back the backend's data actually goes is unknown — untested beyond that range.

3. Fixes/additions in `scraper_daemon.py`

- **Retries every scrape, today's included.** The site itself is flaky enough that a single bare attempt regularly fails — confirmed live, repeatedly, both on the dev machine and on the VM. `_scrape_date_with_retries()` retries up to 4 times, `resume=True` on retries after the first so a late attempt only re-queries commodities the earlier one didn't reach.
- **Auto-detects and backfills gaps in the last 3 days.** `_find_missing_recent_dates()` runs one `SELECT DISTINCT arrival_date` against `fact_market_prices` each cycle. If a day in that window has zero rows — exactly what happened 2026-08-08 through -10 — it's backfilled automatically, right after that cycle's normal today-scrape. Bounded to 3 days on purpose: an unbounded lookback would mean re-attempting some permanent historical gap forever, every cycle.

4. Manual backfill (2026-08-11, dev machine)

Before the daemon-side auto-backfill existed, the 8th/9th/10th/11th were backfilled by hand: a throwaway retry wrapper (`--date=` per day, retried with `--resume` on failure) run locally. Not committed — superseded by the daemon's own retry+backfill logic above.

5. VM deployment

The scraper now also runs as an always-on service on a separate Azure VM (`sethu-admin1` , Ubuntu 24.04, Central India, Standard B2als v2 — 2 vCPU/4GiB), rather than only as a console-window process on the dev Windows PC (whose `start_scraper_daemon.vbs` kept getting silently disabled — suspected AV heuristic flagging a headless-browser-launching scheduled task as malware persistence).

5.1 Prior workload removed

The VM wasn't empty — `~/projects/Sethu_Dashboard` was a live production stack: `nginx` (serving `sethu-admin.farmvaidya.ai`), `sethu-backend.service`, `pipecat-sync.service`, `db-sync.service`, a Node frontend on `:3000`. Confirmed with the project owner it was dead/replaced before removing it: services stopped and disabled, unit files and the `nginx` site config deleted, the project directory (`rm -rf`) removed. Nothing else on the VM depended on any of it.

5.2 Code

Repo is private, so a dedicated **read-only deploy key** (`agmarknet_scraper`, `ed25519`, generated on the VM, added under the repo's Settings → Deploy keys) was used instead of sharing a personal token — scoped to this one repository, pull-only.

```
git clone git@github.com:BondapalliPraneeth/PIPECAT-AGENT-V1.git ~/projects/pipecat
```

5.3 Python environment

`bot_processors/pricing/requirements-scraper.txt` — a lean dependency list (`playwright`, `easyocr`, `asyncpg`, `openpyxl`, `python-dotenv`, `loguru`) instead of the full ~205-line voice-bot requirements file, most of which (`Pipecat`, STT/TTS/LLM SDKs, RAG deps) `scraper_daemon.py`'s own import chain never touches.

Caught during install: `pip` resolved `easyocr`'s unpinned `torch` dependency to the default **GPU/CUDA** build — this VM has no GPU, so that pulled ~3.8GB of unused `nvidia-cu*` / `cuda-toolkit` packages for nothing. Uninstalled those and reinstalled `torch` / `torchvision` from `https://download.pytorch.org/whl/cpu` instead — `venv` went from 5.3GB to 1.5GB, functionally identical (`torch.cuda.is_available()` was always going to be `False` here either way).

Playwright's Chromium: `playwright install --with-deps chromium`. First attempt was run under `sudo`, which downloaded the browser into **root's** cache (`/root/.cache/ms-playwright/`) instead of `azureuser`'s — the service runs as `azureuser`, so it would never have found it. Re-ran without `sudo` (system-level deps were already installed by the first, `--with-deps` pass) to land it in the right place, and removed root's stray copy.

5.4 Database connectivity — Tailscale

`DATABASE_URL` had always pointed at `127.0.0.1:5432` — Postgres running locally on the dev Windows PC. Meaningless from a separate VM. Rather than exposing Postgres to the public internet or migrating it to a managed cloud DB (bigger, separate task), connected the two machines with **Tailscale** (mesh VPN, no port-forwarding, no public exposure):

- Dev PC (`lenova`): `100.109.45.19`
- VM (`agmarknet-scraper-vm`): `100.98.24.27`
- Same Tailscale account (`bondapallipraneeth@`) on both ends.

On the Windows side, `listen_addresses = '*'` was already set in `postgresql.conf`, but `pg_hba.conf` only allowed `127.0.0.1 / ::1`. Added one line scoped tightly to just the VM's IP and just the app's own database/user (not `all / all` like the localhost rules):

```
host    farmvaidya    farmvaidya_app  100.98.24.27/32    scram-sha-256
```

A new Windows Firewall inbound rule opens TCP 5432, but **only** to Tailscale's own address range:

```
New-NetFirewallRule -DisplayName "PostgreSQL from Tailscale (agmarknet scraper)" `
  -Direction Inbound -Protocol TCP -LocalPort 5432 `
  -RemoteAddress 100.64.0.0/10 -Action Allow -Profile Any
```

Config reloaded (`pg_ctl reload` , not a full service restart, so nothing already connected got dropped) rather than restarted.

The VM's `.env` — checked what the scraper's own import chain actually reads first (`os.getenv("DATABASE_URL")`), and nothing else) rather than copying the dev machine's full ~90-key `.env` wholesale:

```
DATABASE_URL=postgresql://farmvaidya_app:***@100.109.45.19:5432/farmvaidya
```

5.5 systemd service

`bot_processors/pricing/agmarknet-scraper.service` — replaces the Windows-only `start_scraper_daemon.vbs` (VBScript doesn't exist on Linux). Installed as `/etc/systemd/system/agmarknet-scraper.service` , `User=azureuser` , `WorkingDirectory=~/projects/pipecat` , `Restart=on-failure` . Enabled (starts on boot), started, confirmed running.

```
sudo cp bot_processors/pricing/agmarknet-scraper.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now agmarknet-scraper
journalctl -u agmarknet-scraper -f # logs
```

6. Verification performed before calling it done

Each layer tested independently, then the whole chain together, before handing it to systemd:

1. **Navigation fix** — form opened, dropdowns picked, a real query returned real rows (today's date).
2. **Date override** — same, with `target_date` set to a past date; returned rows genuinely dated that day, not today.
3. **On the VM specifically:** a scoped one-commodity `scrape_all()` call — real browser launch, real captcha OCR (model auto-downloaded on first use), 68 rows back.
4. **Full chain, VM → Postgres over Tailscale:** `scrape_all()` + `save_scraped()` for one commodity, then an independent `SELECT count(*)` against `fact_market_prices` confirmed the rows actually landed (9 rows for Barley(Jau), dated today).
5. **systemd service:** confirmed `active (running)` , connected to Postgres, progressing through commodities, memory well within the VM's 4GiB (~775MB used, ~2.5GB still free).

7. Known limitations / things not done

- **How far back date-override actually works is unverified** beyond a few days. Don't assume it works for e.g. a month ago without testing that specific date first.
- **DB availability is tied to the dev Windows PC being on.** Tailscale solves the "no port-forwarding" problem, not the "Postgres has to be running somewhere reachable" problem — if that PC is off, the VM's scraper can't write anywhere. Migrating to a managed cloud Postgres would remove this dependency; deferred as a bigger, separate task.
- **db-sync.service / pipecat-sync.service credentials from the deleted Sethu_Dashboard were not recovered before deletion** — if those pointed at a DB that would have solved connectivity more simply, that possibility is gone now.